

CONINT Exercise 3

1. Backend Testing with Jest

```
PS C:\npm test \GIT\evaluated-exercise-7>

> evaluated-exercise-7-example@0.1.0 test
> jest

PASS app/services/isValidEmail.test.ts
PASS app/test/integrationUsers.test.ts

Test Suites: 2 passed, 2 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        4.145 s
Ran all test suites.
PS C:\Users\masix\Documents\GIT\evaluated-exercise-7> █
```

2. Load Testing with k6

```
execution: local
script: k6.test.js
output: -

█ TOTAL RESULTS

checks_total.....: 20923 285.254422/s
checks_succeeded...: 100.00% 20923 out of 20923
checks_failed.....: 0.00% 0 out of 20923

✓ User created
✓ User updated
✓ Users retrieved
✓ Post created
✓ Posts retrieved
✓ Post deleted
✓ User deleted

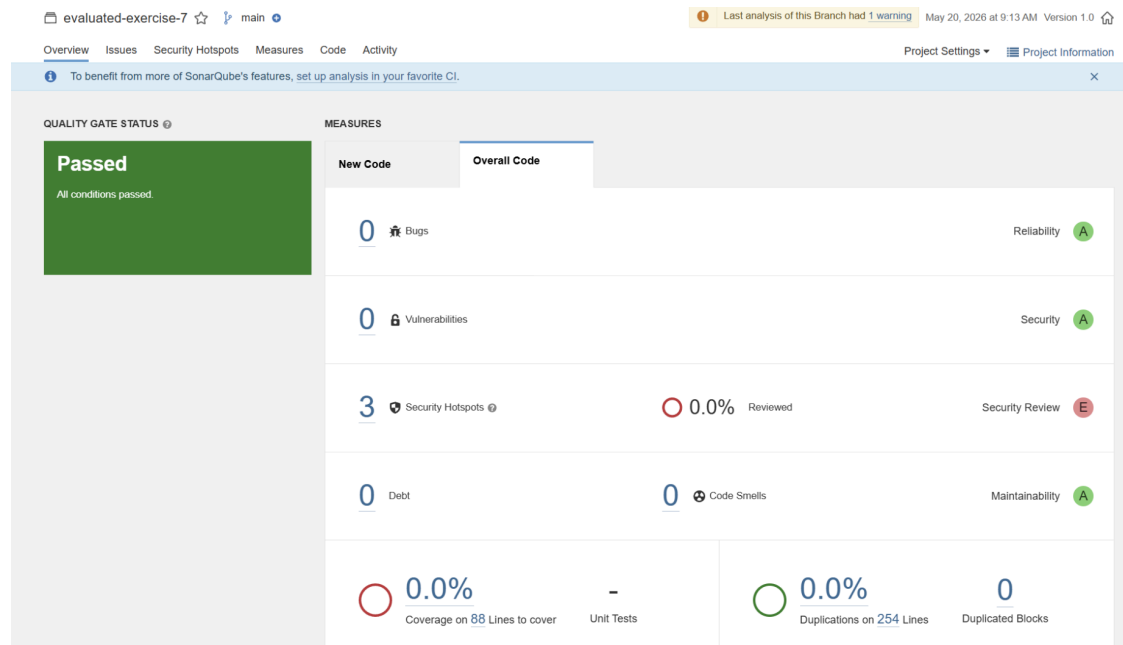
HTTP
http_req_duration.....: avg=613.27ms min=4.07ms med=358.81ms max=2.02s p(90)=1.41s p(95)=1.56s
{ expected_response:true }...: avg=613.27ms min=4.07ms med=358.81ms max=2.02s p(90)=1.41s p(95)=1.56s
http_req_failed.....: 0.00% 0 out of 20923
http_reqs.....: 20923 285.254422/s

EXECUTION
iteration_duration.....: avg=2.72s min=1.02s med=2.88s max=3.44s p(90)=3.22s p(95)=3.37s
iterations.....: 2989 40.750632/s
vus.....: 110 min=110 max=111
vus_max.....: 111 min=111 max=111

NETWORK
data_received.....: 88 MB 1.2 MB/s
data_sent.....: 3.1 MB 43 kB/s

running (1m13.3s), 000/111 VUs, 2989 complete and 0 interrupted iterations
```

3. Sonar Dashboard Screenshots



4. Security Scanning with Snyk

npx snyk auth (hinterlegt lokal einen SNYK-Token und ist nicht immer notwendig)

```
"snyk:auth": "npx snyk auth",
"snyk:scan:dev": "dotenv -e .env.local -- npx snyk test",
"snyk:scan:prod": "npx snyk test -- --no-colors --no-progress --no-updates"
```

```
> evaluated-exercise-7-example@0.1.0 snyk:scan:dev
> dotenv -e .env.local -- npx snyk test

- Analyzing npm dependencies for evaluated-exerc\ Analyzing npm dependencies for evaluated-exerc|
Analyzing npm dependencies for evaluated-exerc

Testing C:\Users\masix\Documents\GIT\evaluated-exercise-7...

Tested 47 dependencies for known issues, found 8 issues, 29 vulnerable paths.

Issues to fix by upgrading:

  Upgrade fastify@5.2.1 to fastify@5.8.5 to fix
  X Use of Less Trusted Source [Medium Severity][https://security.snyk.io/vuln/SNYK-JS-FASTIFY-157
62220] in fastify@5.2.1
    introduced by fastify@5.2.1
  X Allocation of Resources Without Limits or Throttling [Medium Severity][https://security.snyk.i
o/vuln/SNYK-JS-FASTIFY-15182641] in fastify@5.2.1
    introduced by fastify@5.2.1
  X Improper Validation of Specified Type of Input [High Severity][https://security.snyk.io/vuln/S
NYK-JS-FASTIFY-16066793] in fastify@5.2.1
    introduced by fastify@5.2.1
  X Interpretation Conflict [High Severity][https://security.snyk.io/vuln/SNYK-JS-FASTIFY-15182642
] in fastify@5.2.1
    introduced by fastify@5.2.1
  X Improper Validation of Specified Type of Input [High Severity][https://security.snyk.io/vuln/S
NYK-JS-FASTIFY-9788069] in fastify@5.2.1
    introduced by fastify@5.2.1

Issues with no direct upgrade or patch:

  X Regular Expression Denial of Service (ReDoS) [High Severity][https://security.snyk.io/vuln/SNY
K-JS-AJV-15274295] in ajv@8.17.1
    introduced by fastify@5.2.1 > @fastify/ajv-compiler@4.0.2 > ajv@8.17.1 and 5 other path(s)
    This issue was fixed in versions: 6.14.0, 8.18.0
  X Interpretation Conflict [High Severity][https://security.snyk.io/vuln/SNYK-JS-FASTURI-16642394
] in fast-uri@3.0.6
    introduced by fastify@5.2.1 > @fastify/ajv-compiler@4.0.2 > fast-uri@3.0.6 and 8 other path(s)
    This issue was fixed in versions: 3.1.2
  X Directory Traversal [High Severity][https://security.snyk.io/vuln/SNYK-JS-FASTURI-16642399] in
fast-uri@3.0.6
    introduced by fastify@5.2.1 > @fastify/ajv-compiler@4.0.2 > fast-uri@3.0.6 and 8 other path(s)
    This issue was fixed in versions: 3.1.1

Organization:    if25b050
Package manager: npm
Target file:     package-lock.json
Project name:    evaluated-exercise-7-example
Open source:     no
Project path:    C:\Users\masix\Documents\GIT\evaluated-exercise-7
Licenses:        enabled

PS C:\> \GIT\evaluated-exercise-7>
```

5. Configuration Files

a. Unit-Tests

```
app > services > TS isValidEmail.test.ts > ...
1 C:\Users\masix\Documents\GIT\evaluated-exercise-7\app isValidEmail';
2 import { expect, test } from '@jest/globals';
3
4 test('isValidEmail returns true for valid email addresses', () => {
5   expect(isValidEmail('user@example.com')).toBe(true);
6 });
7
8 test('isValidEmail returns false for invalid email addresses', () => {
9   expect(isValidEmail('invalid-email')).toBe(false);
10 });
```

b. Integration-Tests

```
import { expect, test } from "@jest/globals";
Q
test("POST users returns 201 for created user entity", async () => {
  const res = await fetch("http://localhost:5000/api/v1/users", {
    method: "POST",
    headers: {
      Accept: "application/json",
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      name: "Mi Yu",
      email: `miyu${Math.random()}@gmail.com`,
    }),
  });
  expect(res.status).toBe(201);
});

test("POST users returns 400 for invalid email", async () => {
  const res = await fetch("http://localhost:5000/api/v1/users", {
    method: "POST",
    headers: {
      Accept: "application/json",
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      name: "Mi Yu",
      email: `miyu${Math.random()}@.com`,
    }),
  });
  expect(res.status).toBe(400);
});
```

c. K6

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 111,
  duration: '72s',
};

// The default exported function is gonna be picked up by k6 as the entry point for the test script.
export default function () {
  const baseUrl = 'http://host.docker.internal:5000';
  const counter = Math.random().toString(36).substr(2, 9); // Random string to create unique data for

  const userResponse = http.post(baseUrl + '/api/v1/users', JSON.stringify({ name: 'John Doe ' + counter }));
  const user = userResponse.json().user;
  // simulate diverse user interactions
  const responses = http.batch([
    ['PUT', baseUrl + '/api/v1/users/' + user.id, JSON.stringify({ name: user.name + ' Updated' })],
    ['GET', baseUrl + '/api/v1/users'],
    ['POST', baseUrl + '/api/v1/posts', JSON.stringify({ title: 'Post Title ' + counter, content: 'Post Content ' + counter })],
    ['GET', baseUrl + '/api/v1/posts'],
  ]);

  const delPostResponse = http.del(baseUrl + '/api/v1/posts/' + responses[2].json().post.id);
  const delUserResponse = http.del(baseUrl + '/api/v1/users/' + user.id);

  check(userResponse, { 'User created': (r) => r.status === 201 });
  // validate multiple endpoint responses
  check(responses[0], { 'User updated': (r) => r.status === 200 });
  check(responses[1], { 'Users retrieved': (r) => r.status === 200 });
  check(responses[2], { 'Post created': (r) => r.status === 201 });
  check(responses[3], { 'Posts retrieved': (r) => r.status === 200 });
  check(delPostResponse, { 'Post deleted': (r) => r.status === 200 });
  check(delUserResponse, { 'User deleted': (r) => r.status === 200 });

  // Sleep for 1 second to simulate real-world usage
  sleep(1);
}
```

6. Code Samples

```
# Environment variables declared in this file are automatically made available to P
# See the documentation for more detail: https://pris.ly/d/prisma-schema#accessing-

# Prisma supports the native connection string format for PostgreSQL, MySQL, SQLite
# See the documentation for all the connection string options: https://pris.ly/d/co

DATABASE_URL="postgresql://myuser:mypass@database:5432/mydatabase?schema=public"
POSTGRES_USER="myuser"
POSTGRES_PASSWORD="mypass"
POSTGRES_DB="mydatabase"
POSTGRES_HOST="database"
POSTGRES_PORT="5432"
PORT="5000"
HOST="0.0.0.0"
SONAR_TOKEN="sqp_ae9194e83ab92f912535106a6993887494d73e28"
```

a.

```
{ } package.json M X  ⚙️ sonar-project.properties U X  JS jest.config.js

⚙️ sonar-project.properties
1  # The name and key for this project in SonarQube
2  sonar.projectKey=evaluated-exercise-7
3  sonar.projectName=evaluated-exercise-7
4  sonar.projectVersion=1.0
5  # The relative path to the root of your source code
6  sonar.sources=./app
7  # The url to the SonarQube server
8  sonar.host.url=http://localhost:9000
9  # The login token for the SonarQube server
10 sonar.login=${SONAR_TOKEN}
11
```

b.